TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA DEPARTAMENTO DE
SISTEMAS Y COMPUTACIÓN

SEMESTRE 9

Enero - Junio 2026

Ingeniería en Sistemas Computacionales

Patrones de diseño

Abstract Factory

Castillo Lorenzo Gerardo
De Haro Exzacarias Julio Cesar
Del Rio Romero Leonardo Joel
Diaz Morales Katherine Giselle
Rivas Perez Kevin Jonathan

Maribel Guerrero Luis

Tijuana Baja California, a 12 de marzo del 2026

PATRÓN DE DISEÑO
Abstract Factory (Fábrica Abstracta)

Categoría creacional

Fábricas Concretas, con 5 tipos de cocina:

1.- Restaurante Chino
•Plato fuerte: Chow Mein
•Bebida: Té Jazmín
•Postre: Rollito dulce con nieve

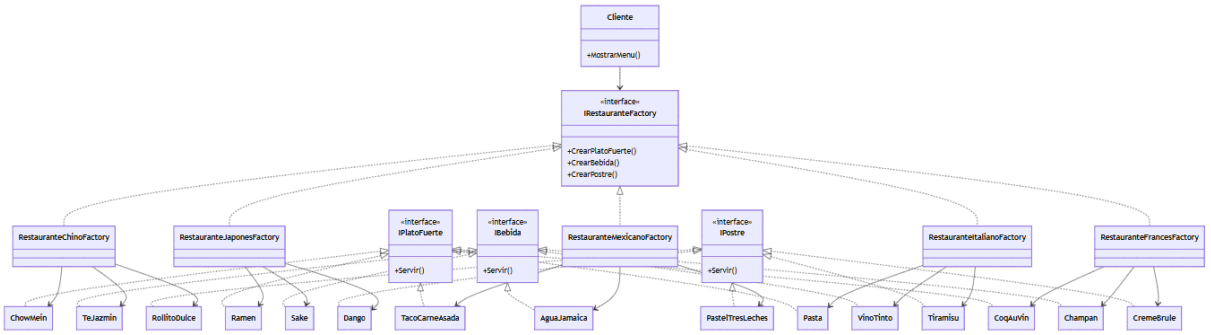
2.- Restaurante Japonés
•Plato fuerte: Ramen
•Bebida: Sake
•Postre: Dango

3.- Restaurante Mexicano
•Plato fuerte: Tacos de carne asada
•Bebida: Agua de Jamaica
•Postre: Pastel de tres leches

4.- Restaurante Francés
•Plato fuerte: Coq au Vin
•Bebida: Champán
•Postre: Crème Brûlée

5.- Restaurante Italiano
•Plato fuerte: Pasta
•Bebida: Vino tinto
•Postre: Tiramisú

Diagrama UML



Código

cliente.cs

```
using System;

namespace FabricasRestaurantes
{
    public class Cliente
    {
        private readonly RestauranteFactory _fabrica;


        public Cliente(RestauranteFactory fabrica)
        {
            _fabrica = fabrica;
        }


        public void GenerarMenu()
        {
            PlatoFuerte plato = _fabrica.CrearPlatoFuerte();
            Bebida bebida = _fabrica.CrearBebida();
            Postre postre = _fabrica.CrearPostre();

            Console.WriteLine("╔═══════════════════════════════════════╗");
            Console.WriteLine("│        MENÚ DEL DÍA          │");
            Console.WriteLine("╠═══════════════════════════════════════╣");
            Console.WriteLine($"│  Plato fuerte : {plato.Servir(),-14}│");
            Console.WriteLine($"│  Bebida       : {bebida.Servir(),-14}│");
            Console.WriteLine($"│  Postre       : {postre.Servir(),-14}│");
            Console.WriteLine("╚═══════════════════════════════════════╝");
        }
    }
}
```

RestauranteFactory.cs

```
namespace FabricasRestaurantes
{

    public abstract class RestauranteFactory
    {
        public abstract PlatoFuerte CrearPlatoFuerte();
        public abstract Bebida CrearBebida();
        public abstract Postre CrearPostre();
    }


    public abstract class PlatoFuerte
    {
        public abstract string Servir();
    }

    public abstract class Bebida
    {
        public abstract string Servir();
    }

    public abstract class Postre
    {
        public abstract string Servir();
    }
}
```

RestauranteChino.cs

```
namespace FabricasRestaurantes
{
    public class RestauranteChino : RestauranteFactory
    {
        public override PlatoFuerte CrearPlatoFuerte() => new ChowMein();
        public override Bebida CrearBebida() => new TeJazmin();
        public override Postre CrearPostre() => new RollitoDulce();
    }

    class ChowMein : PlatoFuerte
    {
        public override string Servir() => "Chow Mein";
    }

    class TeJazmin : Bebida
    {
        public override string Servir() => "Té Jazmín";
    }
```

```
    class RollitoDulce : Postre
    {
        public override string Servir() => "Rollito dulce con nieve";
    }
}
```

RestauranteJapones.cs

```
namespace FabricasRestaurantes
{
    public class RestauranteJapones : RestauranteFactory
    {
        public override PlatoFuerte CrearPlatoFuerte() => new Ramen();
        public override Bebida CrearBebida() => new Sake();
        public override Postre CrearPostre() => new Dango();
    }

    class Ramen : PlatoFuerte
    {
        public override string Servir() => "Ramen";
    }

    class Sake : Bebida
    {
        public override string Servir() => "Sake";
    }

    class Dango : Postre
    {
        public override string Servir() => "Dango";
    }
}
```

RestauranteMexicano.cs

```
namespace FabricasRestaurantes
{
    public class RestauranteMexicano : RestauranteFactory
    {
        public override PlatoFuerte CrearPlatoFuerte() => new TacosCarneAsada();
        public override Bebida CrearBebida() => new AguaJamaica();
        public override Postre CrearPostre() => new PastelTresLeches();
    }

    class TacosCarneAsada : PlatoFuerte
    {
        public override string Servir() => "Tacos de carne asada";
    }

    class AguaJamaica : Bebida
```

```
    {
        public override string Servir() => "Agua de Jamaica";
    }

    class PastelTresLeches : Postre
    {
        public override string Servir() => "Pastel de tres leches";
    }
}
```

RestauranteItaliano.cs

```
namespace FabricasRestaurantes
{
    public class RestauranteItaliano : RestauranteFactory
    {
        public override PlatoFuerte CrearPlatoFuerte() => new Pasta();
        public override Bebida CrearBebida() => new VinoTinto();
        public override Postre CrearPostre() => new Tiramisu();
    }

    class Pasta : PlatoFuerte
    {
        public override string Servir() => "Pasta";
    }

    class VinoTinto : Bebida
    {
        public override string Servir() => "Vino tinto";
    }

    class Tiramisu : Postre
    {
        public override string Servir() => "Tiramisú";
    }
}
```

RestauranteFrances.cs

```
namespace FabricasRestaurantes
{
    public class RestauranteFrances : RestauranteFactory
    {
        public override PlatoFuerte CrearPlatoFuerte() => new CoqAuVin();
        public override Bebida CrearBebida() => new Champan();
        public override Postre CrearPostre() => new CremeBrulee();
    }

    class CoqAuVin : PlatoFuerte
    {
        public override string Servir() => "Coq au Vin";
```

```
    }

    class Champan : Bebida
    {
        public override string Servir() => "Champán";
    }

    class CremeBrulee : Postre
    {
        public override string Servir() => "Crème Brûlée";
    }
}
```

Program.cs

```
using System;
using System.Windows.Forms;

namespace FabricasRestaurantes
{
    internal static class Program
    {
        [STAThread]
        static void Main()
        {
            Application.SetHighDpiMode(HighDpiMode.SystemAware);
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new MainForm());
        }
    }
}
```

MainForm.cs

```
using System;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Windows.Forms;

namespace FabricasRestaurantes
{
    public class MainForm : Form
    {
        private readonly string[]  _nombres  = { "Chino", "Japonés", "Mexicano", "Italiano", "Francés" };
        private readonly string[]  _emojis   = { "🥢",    "🍱",      "🌮",       "🍝",       "🥐"      };
        private readonly Color[]   _primarios =
        {
            Color.FromArgb(190, 30, 45),
            Color.FromArgb( 20, 60,120),
            Color.FromArgb(200, 80,  0),
```

```
    Color.FromArgb(180, 20, 20),
    Color.FromArgb( 25, 50,125),
};
private readonly Color[] _acentos =
{
    Color.FromArgb(255,220,120),
    Color.FromArgb(255,182,193),
    Color.FromArgb(255,220, 50),
    Color.FromArgb(245,245,200),
    Color.FromArgb(255,235,160),
};

private readonly RestauranteFactory[] _fabricas =
{
    new RestauranteChino(),
    new RestauranteJapones(),
    new RestauranteMexicano(),
    new RestauranteItaliano(),
    new RestauranteFrances(),
};

private int _idx = 0;

private Panel  _sidebar   = new Panel();
private Panel  _pnlCard   = new Panel();
private Panel  _pnlHeader = new Panel();
private Label  _lblTitulo    = new Label();
private Label  _lblSubtitulo = new Label();
private Label  _lblPlato     = new Label();
private Label  _lblBebida    = new Label();
private Label  _lblPostre    = new Label();
private Button[] _botones = new Button[5];

public MainForm()
{
    InitializeComponent();
    MostrarMenu(0);
}

private void InitializeComponent()
{
    this.Text           = "Fábrica de Restaurantes";
    this.Size           = new Size(860, 580);
    this.MinimumSize    = new Size(760, 520);
    this.StartPosition  = FormStartPosition.CenterScreen;
    this.BackColor      = Color.FromArgb(22, 22, 32);
    this.Font           = new Font("Segoe UI", 10f);
    this.DoubleBuffered = true;

    _sidebar.Dock      = DockStyle.Left;
    _sidebar.Width     = 210;
    _sidebar.BackColor = Color.FromArgb(30, 30, 42);
```

```
var lblLogo = new Label();
lblLogo.Text      = "🍽  RESTAURANTES";
lblLogo.Font      = new Font("Segoe UI", 10f, FontStyle.Bold);
lblLogo.ForeColor = Color.FromArgb(160, 160, 200);
lblLogo.AutoSize  = false;
lblLogo.Bounds    = new Rectangle(0, 16, 210, 36);
lblLogo.TextAlign = ContentAlignment.MiddleCenter;
lblLogo.BackColor = Color.Transparent;
_sidebar.Controls.Add(lblLogo);

var sep = new Panel();
sep.Bounds    = new Rectangle(14, 58, 182, 1);
sep.BackColor = Color.FromArgb(55, 55, 75);
_sidebar.Controls.Add(sep);

for (int i = 0; i < 5; i++)
{
    int idx = i;
    var btn = new Button();
    btn.Text      = _emojis[i] + "  Restaurante " + _nombres[i];
    btn.TextAlign = ContentAlignment.MiddleLeft;
    btn.Font      = new Font("Segoe UI", 10f);
    btn.ForeColor = Color.FromArgb(190, 190, 220);
    btn.BackColor = Color.FromArgb(38, 38, 54);
    btn.FlatStyle = FlatStyle.Flat;
    btn.Bounds    = new Rectangle(10, 70 + i * 72, 190, 58);
    btn.Cursor    = Cursors.Hand;
    btn.Padding   = new Padding(10, 0, 0, 0);
    btn.FlatAppearance.BorderSize         = 0;
    btn.FlatAppearance.MouseOverBackColor = Color.FromArgb(55, 55, 80);
    btn.Click += (s, e) => MostrarMenu(idx);
    _botones[i] = btn;
    _sidebar.Controls.Add(btn);
}

var lblCredito = new Label();
lblCredito.Text      = "Abstract Factory Pattern";
lblCredito.Font      = new Font("Segoe UI", 7.5f, FontStyle.Italic);
lblCredito.ForeColor = Color.FromArgb(70, 70, 95);
lblCredito.TextAlign = ContentAlignment.MiddleCenter;
lblCredito.Dock      = DockStyle.Bottom;
lblCredito.Height    = 28;
lblCredito.BackColor = Color.Transparent;
_sidebar.Controls.Add(lblCredito);


var pnlMain = new Panel();
pnlMain.Dock      = DockStyle.Fill;
pnlMain.BackColor = Color.FromArgb(22, 22, 32);

_pnlHeader.BackColor = Color.Transparent;
```

```
_pnlHeader.Bounds    = new Rectangle(28, 20, 580, 90);
_pnlHeader.Paint    += PnlHeader_Paint;

_lblTitulo.Text      = "";
_lblTitulo.Font      = new Font("Segoe UI", 22f, FontStyle.Bold);
_lblTitulo.ForeColor = Color.White;
_lblTitulo.AutoSize  = false;
_lblTitulo.Bounds    = new Rectangle(0, 8, 580, 46);
_lblTitulo.BackColor = Color.Transparent;

_lblSubtitulo.Text      = "";
_lblSubtitulo.Font      = new Font("Segoe UI", 10f);
_lblSubtitulo.ForeColor = Color.FromArgb(140, 140, 175);
_lblSubtitulo.AutoSize  = false;
_lblSubtitulo.Bounds    = new Rectangle(2, 54, 580, 26);
_lblSubtitulo.BackColor = Color.Transparent;

_pnlHeader.Controls.Add(_lblTitulo);
_pnlHeader.Controls.Add(_lblSubtitulo);


_pnlCard.BackColor = Color.FromArgb(35, 35, 50);
_pnlCard.Bounds    = new Rectangle(28, 126, 580, 300);
_pnlCard.Paint    += PnlCard_Paint;

string[] iconos = { "🍽️", "🥤", "🍮" };
Label[] lbArr   = new Label[3];
int[]   yPos    = { 42, 142, 242 };

for (int i = 0; i < 3; i++)
{
    var ic = new Label();
    ic.Text      = iconos[i];
    ic.Font      = new Font("Segoe UI Emoji", 20f);
    ic.AutoSize  = false;
    ic.Bounds    = new Rectangle(24, yPos[i] - 2, 42, 42);
    ic.TextAlign = ContentAlignment.MiddleCenter;
    ic.BackColor = Color.Transparent;
    ic.ForeColor = Color.White;

    var lbl = new Label();
    lbl.Text      = "";
    lbl.Font      = new Font("Segoe UI", 11f);
    lbl.ForeColor = Color.White;
    lbl.AutoSize  = false;
    lbl.Bounds    = new Rectangle(76, yPos[i], 460, 42);
    lbl.TextAlign = ContentAlignment.MiddleLeft;
    lbl.BackColor = Color.Transparent;

    _pnlCard.Controls.Add(ic);
    _pnlCard.Controls.Add(lbl);
    lbArr[i] = lbl;
```

```
    }

    _lblPlato  = lbArr[0];
    _lblBebida = lbArr[1];
    _lblPostre = lbArr[2];

    pnlMain.Controls.Add(_pnlHeader);
    pnlMain.Controls.Add(_pnlCard);

    this.Controls.Add(pnlMain);
    this.Controls.Add(_sidebar);

    this.Resize += (s, e) => AjustarLayout();
    AjustarLayout();
}

private void AjustarLayout()
{
    int w = this.ClientSize.Width - 210 - 56;
    if (w < 200) w = 200;
    _pnlHeader.Width    = w;
    _lblTitulo.Width    = w;
    _lblSubtitulo.Width = w;
    _pnlCard.Width      = w;
}

private void MostrarMenu(int idx)
{
    _idx = idx;
    var plato  = _fabricas[idx].CrearPlatoFuerte();
    var bebida = _fabricas[idx].CrearBebida();
    var postre = _fabricas[idx].CrearPostre();

    _lblTitulo.Text      = _emojis[idx] + "  Restaurante " + _nombres[idx];
    _lblTitulo.ForeColor = _acentos[idx];
    _lblSubtitulo.Text   = "Menú del día  ·  cocina " + _nombres[idx].ToLower();

    _lblPlato.Text  = plato.Servir();
    _lblBebida.Text = bebida.Servir();
    _lblPostre.Text = postre.Servir();

    _pnlCard.BackColor = MezclarColor(_primarios[idx], Color.FromArgb(35, 35, 50), 0.15f);

    for (int i = 0; i < 5; i++)
    {
        if (i == idx)
        {
            _botones[i].BackColor = MezclarColor(_primarios[i], Color.FromArgb(38, 38, 54), 0.45f);
            _botones[i].ForeColor = _acentos[i];
            _botones[i].Font      = new Font("Segoe UI", 10f, FontStyle.Bold);
        }
        else
```

```
        {
            _botones[i].BackColor = Color.FromArgb(38, 38, 54);
            _botones[i].ForeColor = Color.FromArgb(190, 190, 220);
            _botones[i].Font      = new Font("Segoe UI", 10f);
        }
    }

    _pnlCard.Invalidate();
    _pnlHeader.Invalidate();
}

private void PnlHeader_Paint(object sender, PaintEventArgs e)
{
    var g = e.Graphics;
    g.SmoothingMode = SmoothingMode.AntiAlias;
    int w = _pnlHeader.Width;
    using var brush = new LinearGradientBrush(
        new Point(0, 0), new Point(Math.Min(w, 340), 0),
        _primarios[_idx], Color.Transparent);
    g.FillRectangle(brush, 0, _pnlHeader.Height - 3, Math.Min(w, 340), 3);
}

private void PnlCard_Paint(object sender, PaintEventArgs e)
{
    var g    = e.Graphics;
    var card = (Panel)sender;
    g.SmoothingMode = SmoothingMode.AntiAlias;

    using var path  = RoundedRect(new Rectangle(0, 0, card.Width - 1, card.Height - 1), 14);
    using var bgBr  = new SolidBrush(card.BackColor);
    g.FillPath(bgBr, path);

    using var penSep = new Pen(Color.FromArgb(55, 55, 75), 1);
    g.DrawLine(penSep, 20, 100, card.Width - 20, 100);
    g.DrawLine(penSep, 20, 200, card.Width - 20, 200);

    using var barBrush = new LinearGradientBrush(
        new Point(0, 16), new Point(0, card.Height - 16),
        _primarios[_idx], _acentos[_idx]);
    g.FillRectangle(barBrush, 0, 16, 5, card.Height - 32);

    using var catFont  = new Font("Segoe UI", 7.5f, FontStyle.Bold);
    using var catBrush = new SolidBrush(_acentos[_idx]);
    g.DrawString("PLATO FUERTE", catFont, catBrush, 76, 14);
    g.DrawString("BEBIDA",       catFont, catBrush, 76, 114);
    g.DrawString("POSTRE",       catFont, catBrush, 76, 214);
}

private static Color MezclarColor(Color a, Color b, float t) =>
    Color.FromArgb(
        (int)(a.R * t + b.R * (1 - t)),
        (int)(a.G * t + b.G * (1 - t)),
```

```
            (int)(a.B * t + b.B * (1 - t)));

        private static GraphicsPath RoundedRect(Rectangle r, int radius)
        {
            var p = new GraphicsPath();
            p.AddArc(r.X,                r.Y,                radius * 2, radius * 2, 180, 90);
            p.AddArc(r.Right - radius * 2, r.Y,                radius * 2, radius * 2, 270, 90);
            p.AddArc(r.Right - radius * 2, r.Bottom - radius * 2, radius * 2, radius * 2,   0, 90);
            p.AddArc(r.X,                r.Bottom - radius * 2, radius * 2, radius * 2,  90, 90);
            p.CloseFigure();
            return p;
        }
    }
}
```

Qué es el patrón utilizado

Es un patrón de diseño de software que se utiliza para proporcionar una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas. Esto se logra mediante la creación de una fábrica abstracta que proporciona una interfaz para crear cada tipo de objeto de la familia, pero deja la implementación concreta de cada objeto a sus subclases.

Su objetivo es proporcionar una interfaz para crear familias de objetos relacionados, en este caso serían platos, bebidas y postres sin especificar sus clases concretas. El fin de este programa es que cada restaurante represente una familia de productos. Todos los productos de una misma familia deben usarse juntos, y el patrón garantiza que no se mezclen productos de diferentes familias.

Cómo se evita el uso de tanto NEW al usar este patrón

Se evita el uso excesivo de new porque la creación de los objetos se centraliza en las fábricas concretas, es decir en lugar de que el cliente instancie directamente cada producto con new, solo crea una única fábrica con un solo new y a través de ella obtiene todos los objetos relacionados como plato, bebida y postre, mediante métodos como CrearPlatoFuerte(). .Además al ser creacional, permite crear familias de objetos dependientes o relacionados sin la necesidad de especificar la familia.

Cómo agregar una nueva cocina

Por ejemplo, en el caso actual para agregar una nueva cocina primero creamos las clases de los productos (plato, bebida y postre) en un nuevo archivo llamado RestauranteArgentino.cs, dentro tendremos la fabrica concreta para Argentina:

public class RestauranteArgentino : RestauranteFactory
(aqui estamos creando una nueva clase que hereda de RestauranteFactory lo que significa que está obligada a implementar los 3 métodos abstractos)

```
  {
     public override PlatoFuerte CrearPlatoFuerte() => new Asado();
     public override Bebida CrearBebida() => new Mate();
     public override Postre CrearPostre() => new Alfajor();
  }
```

Y los productos:

```
  class Asado : PlatoFuerte
  {
     public override string Servir() => "Asado argentino";
  }

  class Mate : Bebida
  {
     public override string Servir() => "Mate amargo";
  }

  class Alfajor : Postre
  {
     public override string Servir() => "Alfajor de dulce de leche";
  }
}
```

Conclusión

Con esta actividad se puede concluir la comprensión de cómo este patrón permite la creación de familias completas de objetos relacionados sin necesidad de tener que especificar sus clases en el código cliente.

Al final resultó que agregar una nueva cocina era más sencillo ya que solo se crea la fábrica y sus productos, sin necesidad de modificar nada de lo que ya funciona y también se comprendió que al centralizar la creación de objetos en las fábricas se evita el uso excesivo de new.

Referencias

Antonio Leiva. (2022, diciembre 4). Abstract Factory - Patrones de diseño. DevExpert. https://devexpert.io/blog/abstract-factory

Gorin, M. (2023, febrero 15). Abstract Factory: One Pattern, Many Clean Solutions. Medium. https://maxim-gorin.medium.com/abstract-factory-one-pattern-many-clean-solutions-d1d4c55f4a04

Refactoring.Guru. (s.f.). Abstract Factory. Recuperado el 11 de marzo de 2026, de https://refactoring.guru/es/design-patterns/abstract-factory

Refactoring.Guru. (2024). Abstract Factory Design Pattern in C#. DoFactory. https://www.dofactory.com/net/abstract-factory-design-pattern